

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓 名：张序鸿

学 院：竺可桢学院

专 业：计算机科学与技术

学 号：3230103265

指导教师：杨莹春

2024 年 12 月 14 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: 流水线处理器—IF、ID 设计与集成

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 马致远

实验地点: 东 4-509 实验日期: 2024 年 12 月 11 日

一、实验目的和要求

- 1.理解流水线 CPU 的基本原理和组织结构
- 2.掌握五级流水线的工作过程和设计方法
- 3.理解流水线取指、译码的设计原理
- 4.设计流水线测试程序

二、实验内容和原理

内容:

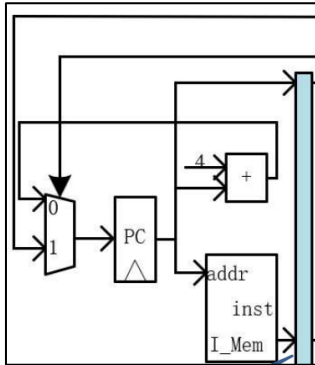
任务一: 设计取指 (IF)、译码 (ID) 模块

- 设计取指模块, 替换 OExp05-1 的取指模块并完成集成
- 设计译码模块, 替换 OExp05-1 的译码模块并完成集成

任务二: 设计 CPU 中断测试方案并完成测试

原理:

2.1 取值 IF 部件



IF: 取指阶段涉及程序计数器 (PC) 和指令存储器 (I_Mem)，程序计数器输出作为地址从指令存储器中读取指令。

IF_reg_ID: 暂存指令、PC，以待下一级使用。

(1) 程序计数器 (Program Counter) :

一个 32 位的寄存器，用于锁存地址，其输出作为访存地址读指令存储器，访存结果为当前指令内容。

```
module REG32(  
    input clk,  
    input rst,  
    input CE,  
    input [31:0]D,  
    output reg [31:0]Q  
);  
  
always @(posedge clk or posedge rst)  
    if (rst==1)  
        Q <= 32'b0;  
    else if (CE == 1)  
        Q <= D;  
endmodule
```

(2) 加法器 (add):

一个输入 32 位的加法器，用于计算 PC+4(顺序执行程序时新的 PC 值)。

```
`timescale 1ns / 1ps  
module add32(  
    input [31:0] a,  
    input [31:0] b,  
    output [31:0] c  
);  
  
assign c = a + b;  
endmodule
```

(3) 32 位多路选择器:

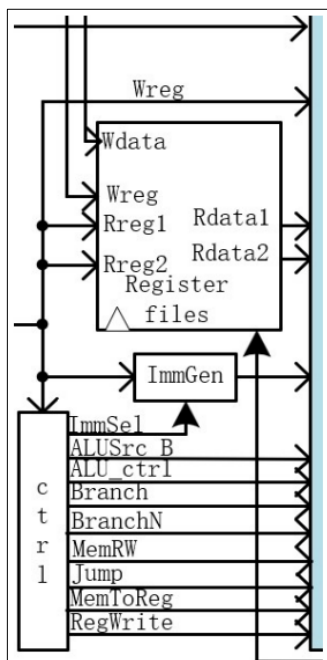
一个 32 位二选一的多路器，用于选择 PC 的更新值，当 PCSrc = 0，选择 PC+4；当 PCSrc = 1，选择跳转目标地址。

```
module MUX2T1_32(  
    input [31:0] I0,  
    input [31:0] I1,  
    input s, // 此处输入 s = PCSrc  
    output [31:0] o  
);  
  
    assign o = s ? I1 : I0;  
endmodule
```

(4) 指令存储器 (I_Mem):

一个异步 ROM，输入地址则输出数据，用于存储 CPU 的指令。在 IF 部件设计时不需要添加，只需要在构建 SOC 时添加。

2.2 译码 ID 部件



ID: 译码阶段涉及寄存器堆 (RegisterFiles) 和译码器 (SCPU_ctrl)、立即数生成单元 (ImmGen)。

从寄存器堆可以读取操作数后，译码器对指令进行解析产生各种各种控制信号，立即数生成单元根据控制信号和输入指令生成各种类型的立即数。

ID_reg_Ex: 暂存 PC 值，寄存器读取的数据 (rs1, rs2)，立即数和控制信号以待下一级使用

(1) 寄存器堆 (RegsFiles):

本质是 32×32bit 寄存器组，指令操作的主要对象；根据寄存器号读取和写入数据，带有写 控制信号。

```

module Regs(
    input clk,
    input rst,
    input [4:0] Rs1_addr,
    input [4:0] Rs2_addr,
    input [4:0] Wt_addr,
    input [31:0]Wt_data,
    input RegWrite,
    output [31:0] Rs1_data,
    output [31:0] Rs2_data,
    output [1023:0] all_regs_data
);
reg [31:0] register [1:31];
integer i;

assign Rs1_data = (Rs1_addr== 0) ? 0 : register[Rs1_addr];
assign Rs2_data = (Rs2_addr== 0) ? 0 : register[Rs2_addr];

always @(posedge clk or posedge rst) begin
    if (rst==1)
        for (i=1; i<32; i=i+1)
            register[i] <= 0;
    else if ((Wt_addr != 0) && (RegWrite == 1))
        register[Wt_addr] <= Wt_data;
end
endmodule

```

(2) 立即数生成单元 (ImmGen):

根据输入指令 (ImmSel) 生成立即数，此处采用拓展后的 ImmGen.

```

module ImmGen_more(
    input wire [2:0] ImmSel,
    input wire [31:0] inst_field,
    output reg [31:0] Imm_out
);

always@*begin
    case(ImmSel)
        3'b000:Imm_out = {inst_field[31:12],12'h000}; // U-type
        3'b001:Imm_out = {{20{inst_field[31]}},inst_field[31:20]};
        //addi\lw(I)
        3'b010:Imm_out =
        {{20{inst_field[31]}},inst_field[31:25],inst_field[11:7]}; //sw(s)
    endcase
end

```

```

        3'b011:Imm_out =
{{20{inst_field[31]}},inst_field[7],inst_field[30:25],inst_field[11:8],
1'b0}; //beq(b)
        3'b100:Imm_out =
{{12{inst_field[31]}},inst_field[19:12],inst_field[20],inst_field[30:21
],1'b0}; //jal(j)
        default: Imm_out = 32'h00000004;
    endcase
end
endmodule

```

(3) 译码控制器 (SCPU_ctrl):

根据输入指令译码输出各个部件的控制信号（指令拓展后版本）。

```

module SCPU_ctrl_more(
    input[4:0]OPcode, //Opcode
    input[2:0]Fun3, //Function
    input Fun7, //Function
    input MIO_ready, //CPU Wait

    output reg [2:0]ImmSel, //立即数选择控制
    output reg ALUSrc_B, //源操作数 2 选择
    output reg [1:0]MemtoReg, //写回数据选择控制
    output reg [1:0]Jump, //jal
    output reg Branch, //beq
    output reg BranchN,
    output reg RegWrite, //寄存器写使能
    output reg MemRW, //存储器读写使能
    output reg [3:0]ALU_Control, //alu 控制
    output reg CPU_MIO
);
reg [1:0] ALUop;
wire [3:0] Fun;
assign Fun = {Fun3, Fun7};
always @(*) begin
    case(OPcode)
        5'b01100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_1_0_0_0_00_10_000; end //ALU
        5'b00000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_01_1_0_0_0_00_00_001; end //load
        5'b01000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_00_0_1_0_0_00_00_010; end //store

```

```

        5'b11000: begin
            case(Fun3)
                3'b000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_0_0_1_0_00_01_011; end // beq
                3'b001: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_0_0_0_1_00_01_011; end // bne
            endcase
        end

        5'b11011: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_01_00_100; end //jump
        5'b00100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_00_1_0_0_0_00_11_001; end //ALU(addi;;;)
        5'b11001: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_10_00_001; end //jarl
        5'b01101: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_11_1_0_0_0_00_00_000; end //lui
        default:
begin {ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} = 14'h0; end
        endcase
    end

    always @(*) begin
        case(ALUop)
            2'b00: ALU_Control = 4'b0010 ;
            2'b01: ALU_Control = 4'b0110 ;
            2'b10:
                case(Fun)
                    4'b0000: ALU_Control = 4'b0010 ; //add
                    4'b0001: ALU_Control = 4'b0110 ; //sub
                    4'b1110: ALU_Control = 4'b0000 ; //and
                    4'b1100: ALU_Control = 4'b0001 ; //or
                    4'b0100: ALU_Control = 4'b0111 ; //slt
                    4'b1010: ALU_Control = 4'b1101 ; //srl
                    4'b1000: ALU_Control = 4'b1100 ; //xor
                    4'b0110: ALU_Control = 4'b1001 ; //sltu
                    4'b0010: ALU_Control = 4'b1110 ; //sll
                    4'b1011: ALU_Control = 4'b1111 ; //sra

```

```

        default: ALU_Control = 4'bx;
    endcase
2'b11:
    case(Fun3)
        3'b000: ALU_Control = 4'b0010 ; //addi
        3'b010: ALU_Control = 4'b0111 ; //slti
        3'b100: ALU_Control = 4'b1100 ; //xori
        3'b110: ALU_Control = 4'b0001 ; //ori
        3'b111: ALU_Control = 4'b0000 ; //andi
        3'b101:
            if(Fun7)
                ALU_Control = 4'b1111 ; //srai
            else
                ALU_Control = 4'b1101 ; //srli
        3'b011: ALU_Control = 4'b1001 ; //sltiu
        3'b001: ALU_Control = 4'b1110 ; //slli
        default: ALU_Control = 4'bx ;
    endcase
endcase
end
endmodule

```


三、实验过程和数据记录

3.1 设计实现取值（IF）模块：

① Pipeline_IF :

PipelineCPU 第一阶段根据 PC 从指令存储器中取出指令并进行 PC 更新。



```
`timescale 1ns / 1ps
module Pipeline_IF(
    input clk_IF,
    input rst_IF,
    input en_IF,
    input [31:0]PC_in_IF,
    input PCSrc,
    output [31:0]PC_out_IF
);

wire [31:0] U0_out , PC_in;
MUX2T1_32 U0(
    .s(PCSrc),
    .I0(PC_in),
    .I1(PC_in_IF),
    .o(U0_out)
);

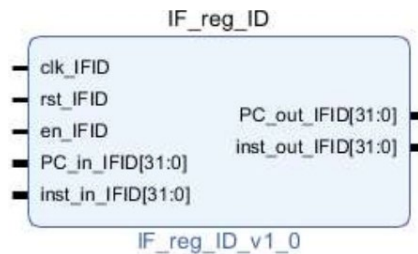
REG32 PC(
    .clk(clk_IF),
    .rst(rst_IF),
    .CE(en_IF),
    .D(U0_out),
    .Q(PC_out_IF)
);

add32 add(
    .a(PC_out_IF),
    .b(32'h4),
    .c(PC_in)
);

endmodule
```

② IF_reg_ID :

PipelineCPU 取值和译码之间的寄存器，寄存 IF 级的输出指令，分割 IF 级和 ID 级的指令或控制信号，防止相互干扰，在 IF 级执行结束时将指令的控制信号传递至下一级。



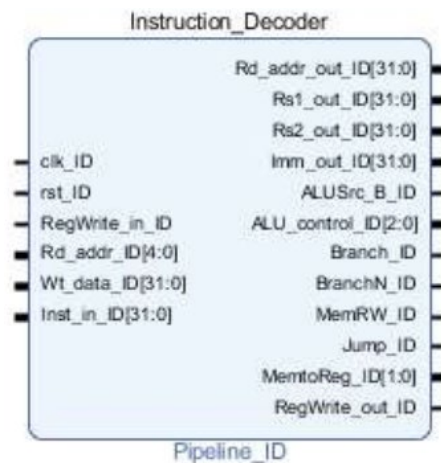
```
`timescale 1ns / 1ps
module IF_reg_ID(
    input clk_IFID,
    input rst_IFID,
    input en_IFID,
    input [31:0]PC_in_IFID,
    input [31:0]inst_in_IFID,
    output reg [31:0]PC_out_IFID,
    output reg [31:0]inst_out_IFID
);

    always @(posedge clk_IFID or posedge rst_IFID)
        if (rst_IFID == 1'b1) begin
            PC_out_IFID <= 32'h0;
            inst_out_IFID <= 32'h0;
        end
        else if (en_IFID == 1'b1) begin
            PC_out_IFID <= PC_in_IFID;
            inst_out_IFID <= inst_in_IFID;
        end
    end
endmodule
```

3.2 设计实现译码（ID）模块：

① Pipeline_ID :

PipelineCPU 第二阶段，通过 SCPU_ctrl 译码之后产生各种控制信号，同时寄存器堆根据所需操作数寄存器的索引读出操作数，立即数生成单元输出所需立即数。



```
`timescale 1ns / 1ps

module Pipeline_ID(
    input clk_ID,
    input rst_ID,
    input RegWrite_in_ID,
    input [4:0]Rd_addr_ID,
    input [31:0]Wt_data_ID,
    input [31:0]Inst_in_ID,

    output [4:0]Rd_addr_out_ID,
    output [31:0]Rs1_out_ID,
    output [31:0]Rs2_out_ID,
    output [31:0]Imm_out_ID,
    output ALUSrc_B_ID,
    output [3:0]ALU_Control_ID,
    output Branch_ID,
    output BranchN_ID,
    output MemRW_ID,
    output [1:0]Jump_ID,
    output [1:0]MemtoReg_ID,
    output RegWrite_out_ID,
    output [1023:0] all_regs_data
);
```

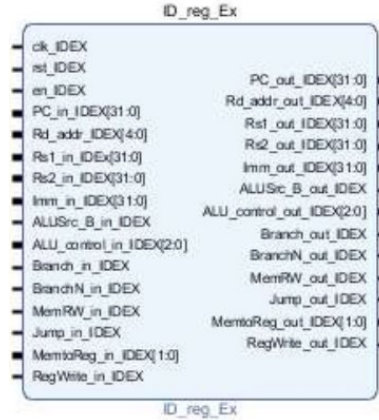
```

wire[2:0] ImmSel;
assign Rd_addr_out_ID = Inst_in_ID[11:7];
Regs Regs_0(
    .clk(clk_ID),
    .rst(rst_ID),
    .Rs1_addr(Inst_in_ID[19:15]),
    .Rs2_addr(Inst_in_ID[24:20]),
    .Wt_addr(Rd_addr_ID),
    .Wt_data(Wt_data_ID),
    .RegWrite(RegWrite_in_ID),
    .Rs1_data(Rs1_out_ID),
    .Rs2_data(Rs2_out_ID),
    .all_regs_data(all_regs_data)
);
ImmGen_more ImmGen_0(
    .ImmSel(ImmSel),
    .inst_field(Inst_in_ID),
    .Imm_out(Imm_out_ID)
);
SCPU_ctrl_more SCPU_ctrl_0(
    .OPcode(Inst_in_ID[6:2]),
    .Fun3(Inst_in_ID[14:12]),
    .Fun7(Inst_in_ID[30]),
    .ImmSel(ImmSel),
    .ALUSrc_B(ALUSrc_B_ID),
    .MemtoReg(MemtoReg_ID),
    .Jump(Jump_ID),
    .Branch(Branch_ID),
    .BranchN(BranchN_ID),
    .RegWrite(RegWrite_out_ID),
    .MemRW(MemRW_ID),
    .ALU_Control(ALU_Control_ID)
);
endmodule

```

② ID_reg_Ex :

PipelineCPU 译码和执行之间的寄存器，存储 ALU 数据和控制信号。寄存 ID 级的输出指令，分割 ID 级和 EX 级的指令或控制信号，防止相互干扰，在 ID 级执行结束时将指令的控制信号传递至下一级。



```
`timescale 1ns / 1ps

module ID_reg_Ex(
    input clk_IDEX, //寄存器时钟
    input rst_IDEX, //寄存器复位
    input en_IDEX, //寄存器使能
    input[31:0] PC_in_IDEX, //PC 输入
    input[4:0] Rd_addr_IDEX, //写目的地址输入
    input[31:0] Rs1_in_IDEX, //操作数 1 输入
    input[31:0] Rs2_in_IDEX, //操作数 2 输入
    input[31:0] Imm_in_IDEX, //立即数输入
    input ALUSrc_B_in_IDEX, //ALU B 输入选择
    input[3:0] ALU_control_in_IDEX, //ALU 选择控制
    input Branch_in_IDEX, //Beq
    input BranchN_in_IDEX, //Bne
    input MemRW_in_IDEX, //存储器读写
    input[1:0] Jump_in_IDEX, //Jal
    input[1:0] MemtoReg_in_IDEX, //写回选择
    input RegWrite_in_IDEX, //寄存器堆读写

    output reg[31:0] PC_out_IDEX, //PC 输出
    output reg[4:0] Rd_addr_out_IDEX, //目的地址输出
    output reg[31:0] Rs1_out_IDEX, //操作数 1 输出
    output reg[31:0] Rs2_out_IDEX, //操作数 2 输出
    output reg[31:0] Imm_out_IDEX, //立即数输出
    output reg ALUSrc_B_out_IDEX, //ALU B 选择
    output reg[3:0] ALU_control_out_IDEX, //ALU 控制
    output reg Branch_out_IDEX, //Beq
```

```

output reg BranchN_out_IDEX, //Bne
output reg MemRW_out_IDEX, //存储器读写
output reg[1:0]Jump_out_IDEX, //Jal
output reg[1:0] MemtoReg_out_IDEX, //写回
output reg RegWrite_out_IDEX //寄存器堆读写
);

always @(posedge clk_IDEX or posedge rst_IDEX) begin
    if(rst_IDEX == 1'b1)begin
        PC_out_IDEX <= 32'h0;
        Rd_addr_out_IDEX <= 5'h0;
        Rs1_out_IDEX <= 32'h0;
        Rs2_out_IDEX <= 32'h0;
        Imm_out_IDEX <= 32'h0;
        ALUSrc_B_out_IDEX <= 1'h0;
        ALU_control_out_IDEX <= 4'h0;
        Branch_out_IDEX <= 1'h0;
        BranchN_out_IDEX <= 1'h0;
        MemRW_out_IDEX <= 1'h0;
        Jump_out_IDEX <= 2'h0;
        MemtoReg_out_IDEX <= 2'h0;
        RegWrite_out_IDEX <= 1'h0;
    end
    else if(en_IDEX == 1'b1) begin
        PC_out_IDEX <= PC_in_IDEX;
        Rd_addr_out_IDEX <= Rd_addr_IDEX;
        Rs1_out_IDEX <= Rs1_in_IDEX;
        Rs2_out_IDEX <= Rs2_in_IDEX;
        Imm_out_IDEX <= Imm_in_IDEX;
        ALUSrc_B_out_IDEX <= ALUSrc_B_in_IDEX;
        ALU_control_out_IDEX <= ALU_control_in_IDEX;
        Branch_out_IDEX <= Branch_in_IDEX;
        BranchN_out_IDEX <= BranchN_in_IDEX;
        MemRW_out_IDEX <= MemRW_in_IDEX;
        Jump_out_IDEX <= Jump_in_IDEX;
        MemtoReg_out_IDEX <= MemtoReg_in_IDEX;
        RegWrite_out_IDEX <= RegWrite_in_IDEX;
    end
end

endmodule

```

四、实验结果分析

~暂无~

五、讨论与心得

原 ppt 的 ID 部分未采用指令拓展,仅添加了 BranchN 信号来实现 bne 指令。为此,我干脆使用了指令拓展后的 ImmGen 和 SCPU_ctrl,并同时修改了 ALU_control、Jump、ImmSel 信号,实现统一。